

计算机操作系统课程设计详细设计文档

首页写明组长和组员姓名学号，以及每个人的分工

组长：_____ 学号：_____

组员及分工：

姓名	学号	分工

一、总体设计

1.1 概述

1.1.1 功能描述

Genshin-OS 是一个基于 Rust 语言开发的微内核操作系统模拟程序。系统严格遵循四层微内核架构，以消息总线（Message Bus）作为所有模块间通信的唯一通道，禁止跨层直接函数调用。用户程序运行在虚拟 CPU 上，通过 INT 0x80 汇编指令触发系统调用，实现进程管理、内存管理、文件 I/O、设备操作等功能。

本系统实现了操作系统课程设计所要求的全部核心功能：

- (1) **进程管理**：支持 `fork()` + `exec()` + `wait()` + `exit()` 的类 Unix 进程生命周期模型。进程状态包括 Creating、Ready、Running、Blocked、Zombie 五种，PID 1 为系统 init 进程。支持信号量同步（`sem_wait` / `sem_signal`）和互斥锁（`lock_acquire` / `lock_release`）。
- (2) **处理机调度**：采用双核 SMP Round-Robin 调度算法，时间片为 3 个时钟周期。硬件 Timer 以 100Hz 频率产生中断，通过 `crossbeam_channel::select!` 机制驱动 ProcessService 的主事件循环。每个 CPU 独立调度，通过 HashSet 去重保证同一时刻不同 CPU 运行不同进程。
- (3) **内存管理**：基于 4KB 页式存储管理，支持按需调页（Demand Paging）和交换（Swapping）。物理内存通过 FrameAllocator 以帧为单位分配和回收，虚拟地址经 MMU 转换为物理地址。缺页中断由 MemoryService 处理，支持页换入/换出到交换分区。
- (4) **文件系统**：实现虚拟文件系统（VFS），以树形目录结构组织文件。VFS 元数据通过 JSON 持久化，文件内容存储于虚拟磁盘镜像 `.genshin-disk.img` 中。支持 `ls`、`mkdir`、`touch`、`cat`、`write`、`rm`、`stat` 等文件操作命令。
- (5) **设备管理**：DeviceService 直接订阅消息总线，提供剪贴板设备。设备的申请、使用、释放通过 `device_open` / `device_close` 系统调用来完成。
- (6) **中断机制**：实现 Timer 中断（100Hz 时钟中断驱动进程调度）、缺页中断（Page-Fault，由 MMU 触发）、系统调用陷阱（SyscallTrap，INT 0x80 触发）。中断经 Kernel 路由至对应的服务通道。
- (7) **UI 界面**：提供交互式命令行 Shell（支持 20+ 命令）和全屏 TUI 进程监控面板 `pmon`（仿 Linux `htop`，展示进程状态、内存用量、磁盘用量的实时动态变化）。

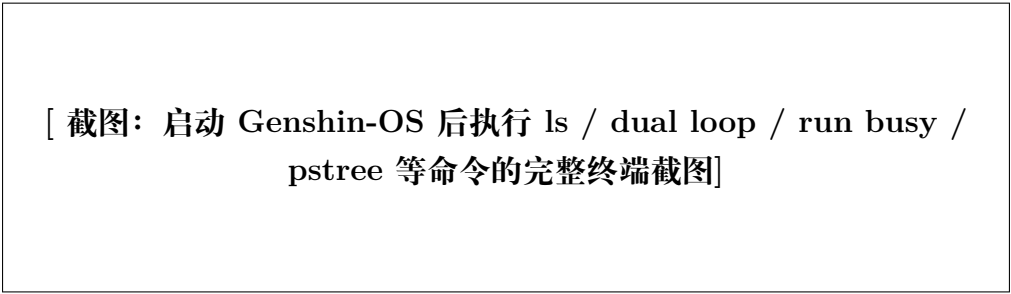


图 1: Genshin-OS Shell 交互界面

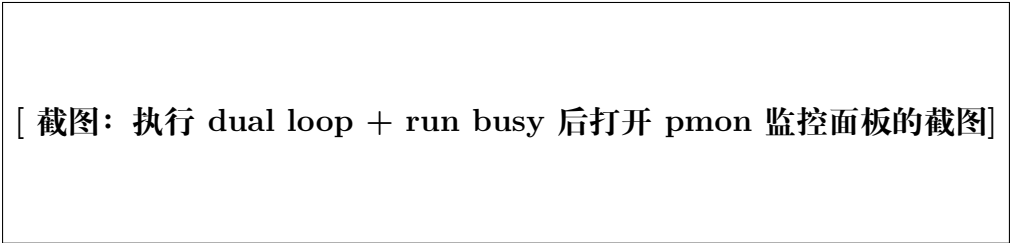


图 2: pmon TUI 进程/内存/磁盘监控面板

系统的总体架构如图 3 所示，采用四层微内核模型。

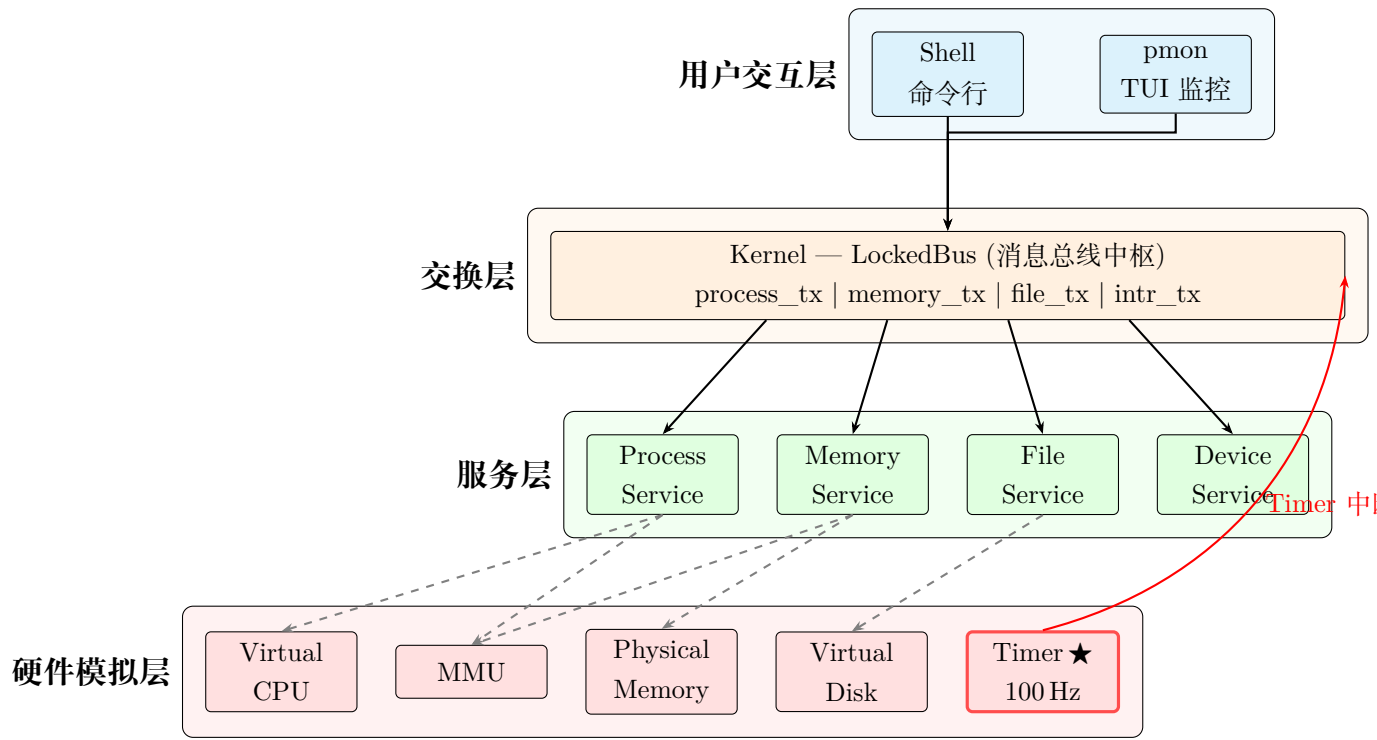


图 3: 系统总体架构图——四层微内核模型

1.1.2 运行环境

- 操作系统：Linux (Ubuntu / Debian / Arch)，亦支持 Windows 和 macOS (需 Rust 工具链)

- 运行时依赖：无（编译为单二进制可执行文件 `chao-os`）
- 终端要求：支持 ANSI 转义序列和 Raw Mode 的终端模拟器（`pmon` TUI 所需）

1.1.3 开发环境

- 编程语言：Rust (edition 2024)
- 编译器：rustc 1.94.0+
- 构建工具：Cargo
- 版本控制：Git
- 主要依赖：
 - `crossbeam-channel` 0.5 —消息总线通道
 - `ratatui` 0.29 + `crossterm` 0.28 —TUI 终端渲染（`pmon`）
 - `serde` + `serde_json` —VFS 元数据持久化
 - `thiserror` 2.0 —错误类型派生